1. Introduction

This project is a **Student Report Card System** developed using the **C++ programming language.** It follows a procedural programming approach and provides a menu-driven interface to manage student academic records. File handling is used to ensure permanent data storage.

2. Objectives

- To store and manage student records digitally
- To reduce manual record keeping
- To demonstrate core C++ programming concepts

3. Description of the System

The system stores roll number, name, and marks using arrays. Users can add, view, edit, and delete records through a menu-driven interface. All data is saved in a text file for future use.

4. Programming Approach

The project is implemented using a procedural programming approach. Functions, arrays, loops, conditional statements, and file handling are used.

5. Program Logic (Pseudocode)

```
START
Load data from file
Display Menu
Take user choice
Perform selected operation
Save data to file
Repeat until Exit
END
```

06.IPO CHART

INPUTS	Process	Output
1. Add Students :		
Roll No (integer), Name (string), Marks (float)	1. Check if database is full (total ≥ SIZE) 2. Validate marks (0 ≤ marks ≤ 1100) 3. Add student to database (rollNos, names, marks arrays)	Student Records (rollNo, name, marks, percentage) or No Records!
2. Show Students :		
None	1. Check if records exist (total > 0) 2. Display student records (rollNo, name, marks, percentage)	Student Records (rollNo, name, marks, percentage) or No Records!
3. Edit Student :		
Roll No (integer), New Roll No (integer), New Name (string), New Marks (float).	1. Search for student (rollNo) 2.if found then update it 3. Validate new marks (0 ≤ marks ≤ 1100) 4.Update student details (rollNos, names, marks arrays) .	Record Updated! or Student Not Found!
4. Delete Student :		
Roll No (integer).	1. Search for student (rollNo) 2. Delete student from database (rollNos, names, marks arrays).	Student Deleted! or Student Not Found!
5. Exist:		
None	1. Display "Exist program!" message. 2. Exit the program or loop.	Exist program!
6. Main Menu:		
Choice (1-5)	1. Display menu. 2. Perform action based on choice (add/ show/ edit/ delete/ exit)	Perform Action or Exist program!

7. Functions Used in the Program

1: addStudent()

This function is used to add a new student record. It takes roll number, name, and marks as input. Marks are validated to ensure they are between 0 and 1100.

2: showStudents()

This function displays all student records. It also calculates and displays the percentage of each student.

3: editStudent()

This function allows the user to edit an existing student record by entering the roll number.

4: deleteStudent()

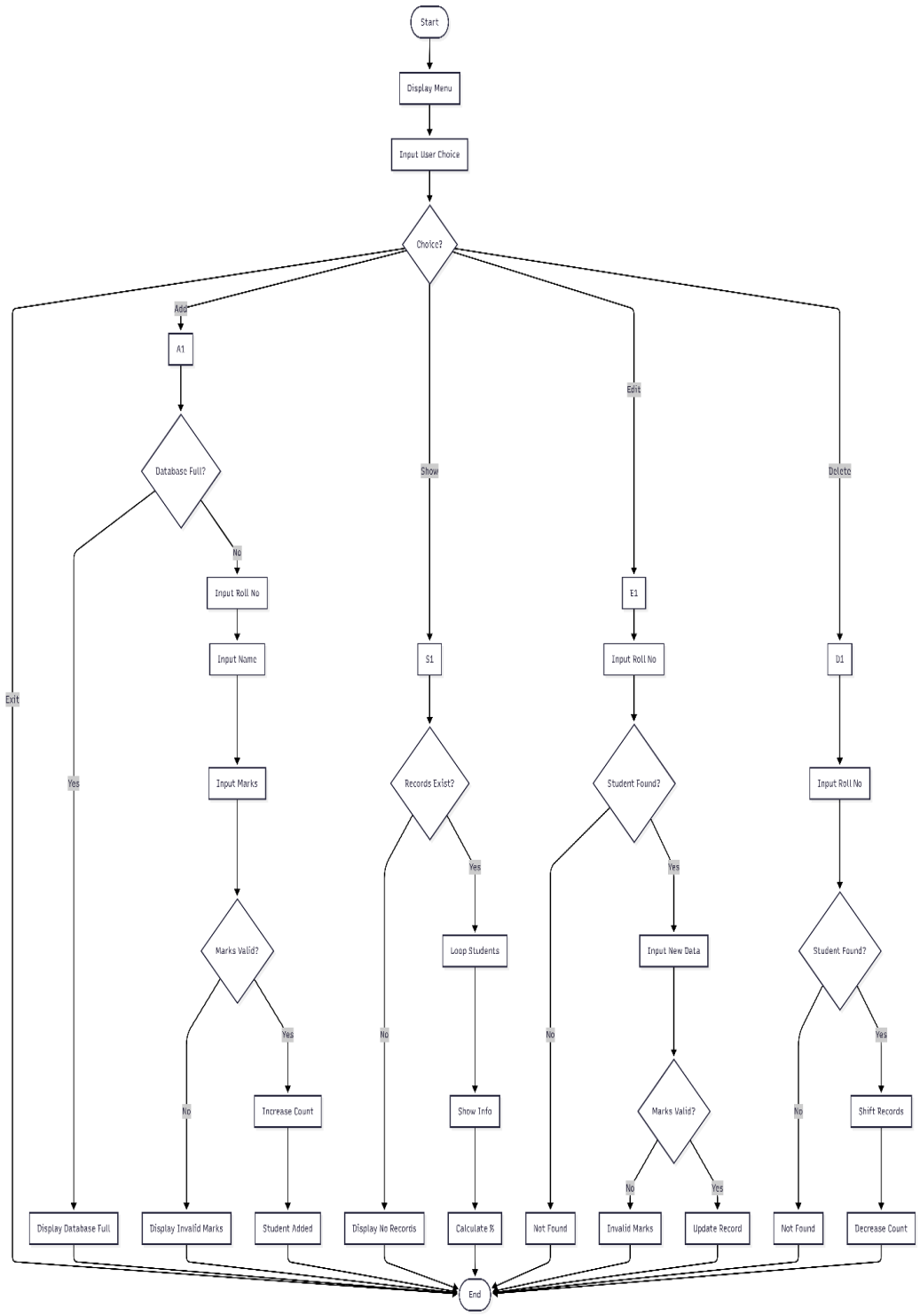
This function deletes a student record based on the roll number and shifts remaining records.

5: main()

The main function displays a menu with options:

1. Add Student
2. Show Students
3. Edit Student
4. Delete Student
5. Exit

08.Flowchart


Start
Display Menu
Input User Choice
Choice?
Add
Show
Edit
Delete
Exit
A1
Database Full?
Yes
No
Input Roll No
Input Name
Input Marks
Marks Valid?
No
Yes
Increase Count
Display Database Full
Display Invalid Marks
Student Added
S1
Records Exist?
No
Yes
Loop Students
Show Info
Display No Records
Calculate %
E1
Input Roll No
Student Found?
No
Yes
Input New Data
Marks Valid?
No
Yes
Not Found
Invalid Marks
Update Record
D1
Input Roll No
Student Found?
No
Yes
Shift Records
Not Found
Decrease Count
End

09.Program

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

const int SIZE = 100;
int rollNos[SIZE];
string names[SIZE];
float marks[SIZE];
int total = 0;
const string FILE_NAME = "students.txt";

// Load data from file
void loadData() {
    ifstream file(FILE_NAME);
    if (!file) return;
    while (file >> rollNos[total] && total < SIZE) {
        file.ignore(); // Ignore newline
        getline(file, names[total]);
        file >> marks[total];
        total++;
    }
    file.close();
}

// Save data to file
void saveData() {
    ofstream file(FILE_NAME);
    for (int i = 0; i < total; i++) {
        file << rollNos[i] << endl;
        file << names[i] << endl;
        file << marks[i] << endl;
    }
    file.close();
}

// Function to Add Students:
void addStudent() {
    if (total >= SIZE) {
        cout << "Database Full!\n";
        return;
    }
```

```
    cout << "Enter Roll No: ";
    cin >> rollNos[total];
    cout << "Enter Name: ";
    cin.ignore();
    getline(cin, names[total]);
    while (true) {
        cout << "Enter Marks Out Of 1100: ";
        cin >> marks[total];
        if (marks[total] >= 0 && marks[total] <= 1100) {
            break;
        } else {
            cout << "Invalid marks! Please enter marks between 0 and
1100.\n";
        }
    }
    total++;
    saveData(); // Save after adding
    cout << "Student Added!\n";
}

// Function to Show Students:
void showStudents() {
    if (total == 0) {
        cout << "No records!\n";
        return;
    }
    for (int i = 0; i < total; i++) {
        float percentage = (marks[i] / 1100) * 100;
        cout << "\nRoll No: " << rollNos[i];
        cout << "\nName: " << names[i];
        cout << "\nTotal Marks: 1100";
        cout << "\nMarks Obtained: " << marks[i];
        cout << "\nPercentage: " << percentage << "%\n";
    }
}

// Function to Edit students:
void editStudent() {
    int r;
    cout << "Enter Roll No to Edit: ";
    cin >> r;
    for (int i = 0; i < total; i++) {
        if (rollNos[i] == r) {
            cout << "Enter New Roll No: ";
```

```
            cin >> rollNos[i];
            cout << "Enter New Name: ";
            cin.ignore();
            getline(cin, names[i]);
            while (true) {
                cout << "Enter New Marks: ";
                cin >> marks[i];
                if (marks[i] >= 0 && marks[i] <= 1100) {
                    break;
                } else {
                    cout << "Invalid marks! Please enter marks between 0
and 1100.\n";
                }
            }
            saveData(); // Save after editing
            cout << "Record Updated!\n";
            return;
        }
    }
    cout << "Student Not Found!\n";
}

// Function to delete students:
void deleteStudent() {
    int r;
    cout << "Enter Roll No to Delete: ";
    cin >> r;
    for (int i = 0; i < total; i++) {
        if (rollNos[i] == r) {
            for (int j = i; j < total - 1; j++) {
                rollNos[j] = rollNos[j + 1];
                names[j] = names[j + 1];
                marks[j] = marks[j + 1];
            }
            total--;
            saveData(); // Save after deleting
            cout << "Student Deleted!\n";
            return;
        }
    }
    cout << "Student Not Found!\n";
}

int main() {
```

```
    loadData(); // Load data on startup
    int choice;
    do {
        cout<<endl;
        cout<<"     STUDENT REPORT CARD SYSTEM"<<endl;
        cout << "\n1. Add\n2. Show\n3. Edit\n4. Delete\n5. Exit\nChoice: ";
        cin >> choice;
        if (choice == 1) addStudent();
        else if (choice == 2) showStudents();
        else if (choice == 3) editStudent();
        else if (choice == 4) deleteStudent();
        else if (choice == 5) cout << "Exit Program!\n";
        else cout << "Invalid choice!\n";
    } while (choice != 5);
    return 0;
}
```

Output:

Add Student:

```
1. Add
2. Show
3. Edit
4. Delete
5. Exit
Choice: 1
Enter Roll No: 01
Enter Name: Ali
Enter Marks Out Of 1100: 888
Student Added!
```

Show

```
1. Add
2. Show
3. Edit
4. Delete
5. Exit
Choice: 2

Roll No: 1
Name: Ali
Total Marks: 1100
Marks Obtained: 888
Percentage: 80.7273%
```

Edit

```
1. Add
2. Show
3. Edit
4. Delete
5. Exit
Choice: 3
Enter Roll No to Edit: 01
Enter New Roll No: 2
Enter New Name: azhar
Enter New Marks: 999
Record Updated!
```

Delete

```
1. Add
2. Show
3. Edit
4. Delete
5. Exit
Choice: 4
Enter Roll No to Delete: 2
Student Deleted!
```

Exist

```
1. Add
2. Show
3. Edit
4. Delete
5. Exit
Choice: 5
Exist Program!
```

10: Output Description

The program displays confirmation messages such as:

- Student Added
- Record Updated
- Student Deleted
- Student Not Found

It also displays student details including roll number, name, marks, and percentage.

11.Advantages

• Easy to use
• Beginner friendly
• Permanent data storage
• Efficient record handling

12.Limitations

• Console-based system
• Fixed-size arrays
• No graphical interface

13.Conclusion

The Student Report Card System is a reliable procedural C++ application that demonstrates fundamental programming concepts and is suitable for academic submission.